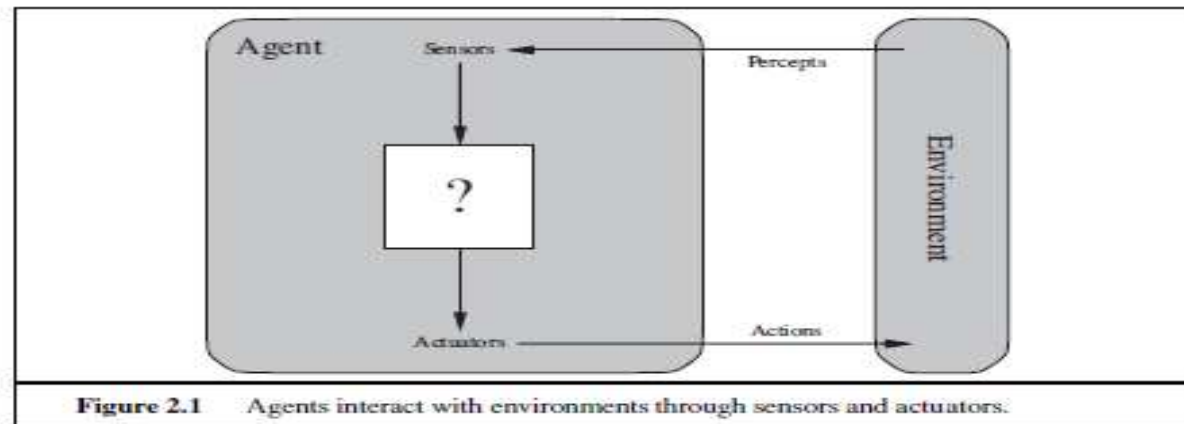


Agent

- An agent perceives the environment through sensors and acts upon the environment through actuators

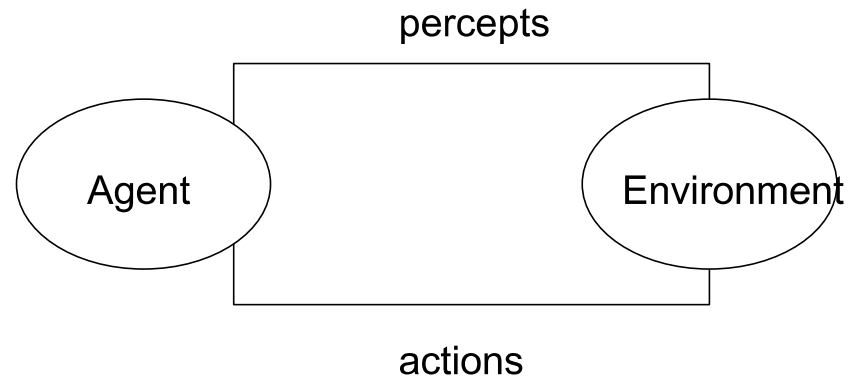


From Russel and Norvig, "Artificial Intelligence : A Modern Approach"

Agents (contd.)

- A human agent has ears, eyes and other organs for sensors and legs, hand, voice etc. for actuators
- A robotic agent has camera, infrared range finders etc. as sensors and various motors for actuators
- **Percept** is the perceptual input of the agent
- **Percept sequence** is the history of everything the agent has perceived
- Agents choice of action depends on the entire percept sequence to date
- Agent's behaviour is described by the **agents function** which maps the percept behaviour to an action
- This can be implemented as a **table look-up**

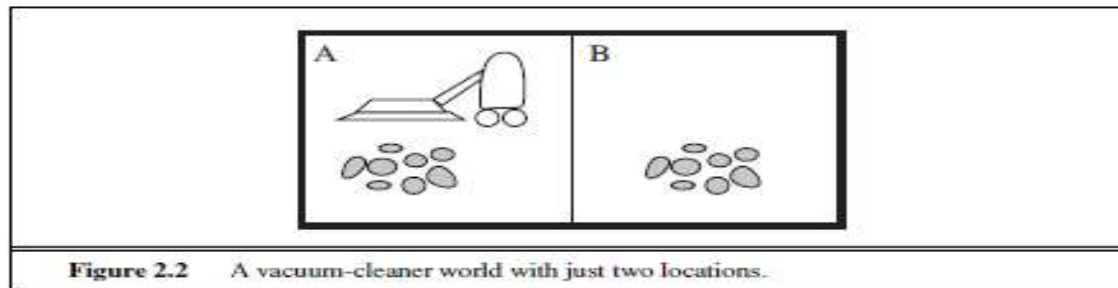
Intelligent Agent



Sensors for percepts
Human: eyes, nose, mouth, skin, ears
Agent : camera, infrared range finders etc.

Effectors for actions
Human: hands, legs, mouth
Agent : motors

Example : vacuum cleaner



Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
⋮	⋮
[A, Clean], [A, Clean], [A, Clean]	Right
[A, Clean], [A, Clean], [A, Dirty]	Suck
⋮	⋮

Figure 2.3 Partial tabulation of a simple agent function for the vacuum-cleaner world shown in Figure 2.2.

Rationality

- Rational agent is one that does the right thing
- Right thing means the sequence of actions done by agent at that time is desirable
- A performance measure evaluates the desirability
- Performance measures are designed according to what one want in the environment

Rationality (contd)

What is rational at a given time depends on

1. The performance measure that defines the criterion of success
2. The agent's prior knowledge of the environment
3. The actions that the agent can perform
4. The agent's percept sequence to date

Definition of a rational agent

“For each possible percept sequence, a rational agent should select an action that is expected to maximise its performance measure, given the evidence provided by the percept sequence, and whatever build-in knowledge the agent has”

Autonomy

- If the agent depends on the prior knowledge of its designer rather than on its own percepts, the agent lacks autonomy
- A rational agent should be autonomous – it should learn what it can to compensate for partial or incorrect prior knowledge
- E.g. a vacuum cleaning agent which learns to foresee where and when additional dirt will appear will do better than one that does not
- At the start, the agent does not have complete autonomy
- When the agent has little or no experience, it would have to act randomly unless the designer gave some assistance
- Just as evolution provides animals with just enough build-in reflexes to survive long enough to learn for themselves, an agent has some initial knowledge as well as the ability to learn
- After sufficient experience of its environment, the behaviour of a rational agents can become independent of its prior knowledge
- The incorporation of learning allows one to design a rational agent that will succeed in a vast variety of environments

Specifying the task environment

- This is called PEAS(Performance, Environment, Actuators, Sensors) description
- First specify the task environment as fully as possible

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

Figure 2.4 PEAS description of the task environment for an automated taxi.

PEAS description for some tasks

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	Correct image categorization	Downlink from orbiting satellite	Display of scene categorization	Color pixel arrays
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, operators	Valves, pumps, heaters, displays	Temperature, pressure, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, suggestions, corrections	Keyboard entry

Figure 2.5 Examples of agent types and their PEAS descriptions.

Properties of task environments

1. Fully observable vs Partially observable
2. Single agent vs Multiagent
3. Deterministic vs Stochastic
4. Episodic vs Sequential
5. Discrete vs Continuous
6. Known vs Unknown

Properties of task environments

Fully observable vs Partially observable

- ❖ If the agent's sensor gives it access to the complete state of the environment at each point in time, the task environment is fully observable
- ❖ A task environment is fully observable if the sensors detect all aspects that are relevant to the choice of action
- ❖ In fully observable environments, the agent need not maintain any internal state to keep track of the world
- ❖ An environment might be partially observable because of noisy or inaccurate sensors or because parts of the state are simply missing from the sensor data
 - E.g. if vacuum cleaner agent has only local dirt sensor and cannot say if there is dirt in other rooms
 - E.g. automated taxi will not be able to sense the entire environment
- ❖ If the agent has no sensors at all then the environment is unobservable

Properties (contd)

Single agent vs Multiagent :

- ❖ Does an agent A (e.g. taxi driver) have to view object B (another vehicle) as an agent or as an object?
- ❖ If B is maximizing its performance metric (e.g. in chess) which minimizes A's performance metric, it is a competitive multiagent environment. Therefore, chess is a competitive multiagent environment
- ❖ In taxi driving agent, avoiding collisions maximizes the performance metric of all agents, so it is a partially cooperative multiagent environment. It is also partially competitive as, e.g. only one vehicle can occupy a parking space

Properties (contd.)

Deterministic vs Stochastic :

- ❖ If the next state of the environment is completely determined by the present state and the action executed by the agent, environment is deterministic otherwise it is stochastic
- ❖ An agent need not worry about uncertainty in a fully observable deterministic environment
- ❖ If the environment is partially observable, most often it is stochastic
- ❖ E.g. Taxi-driving is stochastic as we cannot predict the behaviour of traffic. Also the tire may become flat or the engine can seize up without warning.
- ❖ The vacuum world is deterministic unless we introduce stochastic elements like dust appearing randomly or an unreliable suction mechanism
- ❖ An environment is uncertain if it is not fully observable or not deterministic

Properties (contd.)

Episodic vs Sequential :

- ❖ In an episodic task environment, the agent's experience is divided into atomic episodes
- ❖ In each episode, the agent receives a percept and performs a single action
- ❖ The next episode does not depend on the actions taken in previous episodes
- ❖ E.g. Classification tasks
- ❖ E.g. an agent that has to spot defective parts in an assembly line. Each decision is based on the current part regardless of previous decisions. The current decision doesn't affect whether the next part is defective
- ❖ E.g. chess and taxi-driving are sequential. Short-term actions can have long term consequences
- ❖ Episodic environments are much simpler than sequential as the agent does not need to think ahead

Properties (contd.)

Discrete vs Continuous :

- ❖ This applies to the state of the environment, to the way time is handled and to the percepts and actions of the agent
- ❖ E.g. Chess has a finite number of discrete states and discrete set of percepts and actions
- ❖ E.g. Taxi driving is a continuous state and continuous time problem. The speed and location of the taxi and other vehicles sweep through a range of continuous values smoothly over time. Actions are also continuous. Input from digital cameras is of continuously varying intensities and locations

Properties (contd.)

Known vs Unknown :

- ❖ In a known environment, the outcome for all actions are given.
- ❖ If the environment is unknown, the agent will have to learn how it works in order to make good decisions
- ❖ It is possible for a known environment to be partially observable e.g. in solitaire card game, I may know the rules but do not know the cards not yet turned over
- ❖ Conversely, an unknown environment can be fully observable e.g. in a new video game, the screen may show the entire game state, but I still don't know what the buttons do until I try them

Example of task environment characteristics

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete
Figure 2.6 Examples of task environments and their characteristics.						

The hardest case is partially observable, multiagent, stochastic, sequential, dynamic, continuous and unknown as in case of taxi driving

Structure of agents

- AI designs an agent program which implements the agent function i.e. the mapping from percept sequence to action
- The program will run on a computer system with sensors and actuators which is the architecture .
- ***Agent = Architecture + Program***
- An agent program takes the current percept from the sensor and returns an action to the actuator

Table-driven Agent

```
function TABLE-DRIVEN-AGENT(percept) returns an action
  persistent: percepts, a sequence, initially empty
               table, a table of actions, indexed by percept sequences, initially fully specified

  append percept to the end of percepts
  action ← LOOKUP(percepts, table)
  return action
```

Figure 2.7 The TABLE-DRIVEN-AGENT program is invoked for each new percept and returns an action each time. It retains the complete percept sequence in memory.

From Russel and Norvig, "Artificial Intelligence : A Modern Approach"

Table-driven Agent (Contd.)

- The table-driven approach to table construction is doomed to failure
- If P is the set of possible percepts and T the lifetime of the agent, the look-up

table will contain $\sum_{t=1}^T |P|^t$ entries

- In the automated taxi, the visual input from a single camera comes in at the rate of 27 mega bytes per second (30 frames per second, 640 X 480 pixels with 24 bits of colour information) giving a lookup table with over $10^{250,000,000,000}$ entries for an hours driving
- Even chess would have 10^{150} entries
- The daunting size of these tables means that
 - a. No physical agent in this universe will have the space to store the table
 - b. The designer wouldn't have time to create the table
 - c. No agent could ever learn the right table entries from its experience
 - d. The designer has no guidance about how to fill in the table entries

Kinds of agent programs

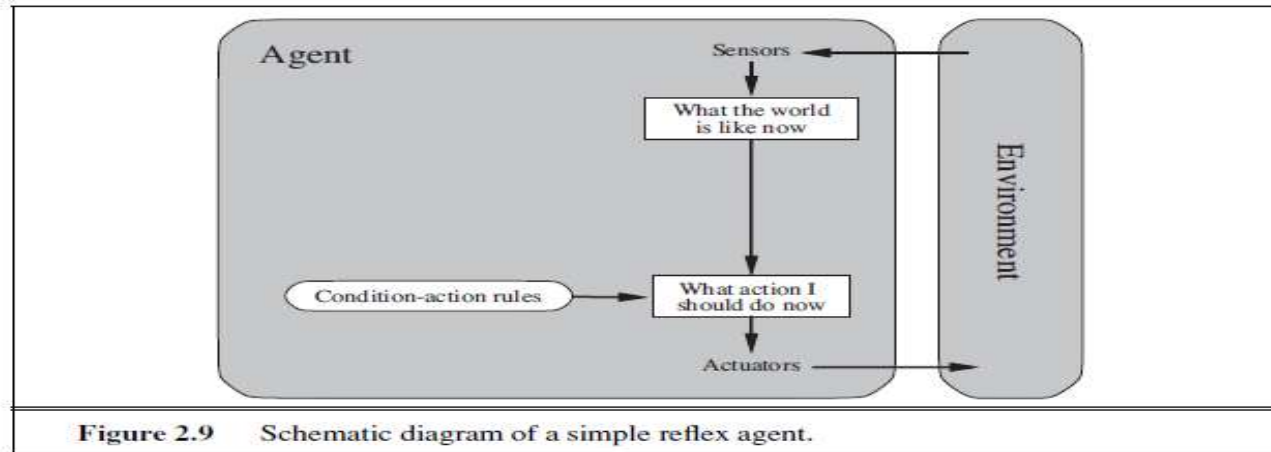
- Four kinds
 - Simple reflex agents
 - Model-based reflex agents
 - Goal-based agents
 - Utility-based agents

Simple reflex agent

- Simplest agent
- Agent selects action based on the basis of the current percept, ignoring the rest of percept history
- The condition-action rules allows agent to make the connection from percept to action
- Rectangles denote the current internal states of the agent's decision process
- Ovals represent the background information used in the process
- This agent will work if the correct decision can be made on the basis of only the current percept
- This works only if the environment is fully observable

EX: Automatic doors, motion sensor lights,

Simple reflex agent (contd.)



```
function SIMPLE-REFLEX-AGENT(percept) returns an action  
persistent: rules, a set of condition-action rules  
  
  state ← INTERPRET-INPUT(percept)  
  rule ← RULE-MATCH(state, rules)  
  action ← rule.ACTION  
  return action
```

Figure 2.10 A simple reflex agent. It acts according to a rule whose condition matches the current state, as defined by the percept.

Model-based reflex agents

- Agent keeps track of a part of the world it cannot see i.e. agent maintains an internal state which depends on the percept history and therefore reflects at least some of the unobserved aspects of the current state
- Updating internal state information as time goes on requires two types of knowledge
 - Information of how the world evolves independently of the agent
 - Information of how the agent's own action affects the world
- How the world works requires a model of the world and is therefore a model-based agent
- UPDATE_STATE creates new internal state description
- Can be used in partially observable environments

Ex: Robot Vacuum Cleaner

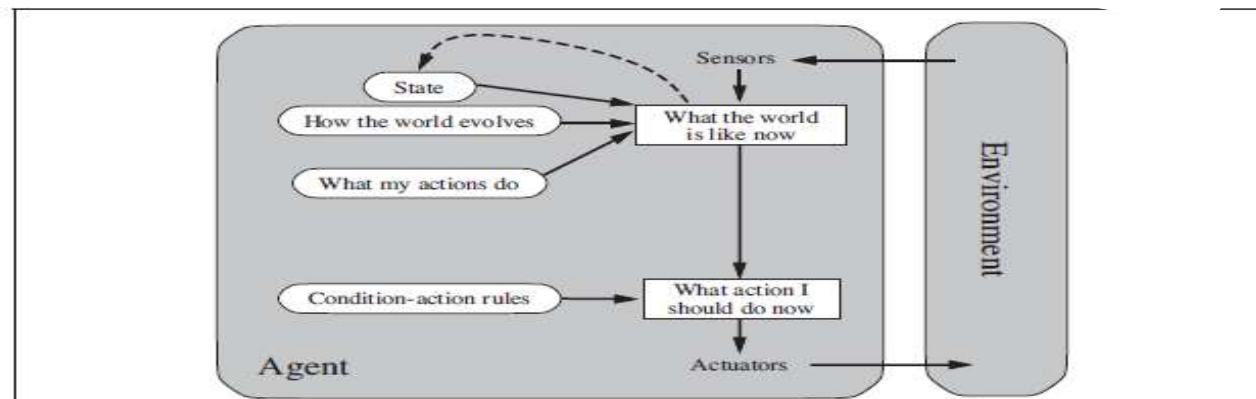


Figure 2.11 A model-based reflex agent.

```

function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
               model, a description of how the next state depends on current state and action
               rules, a set of condition-action rules
               action, the most recent action, initially none

  state ← UPDATE-STATE(state, action, percept, model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action

```

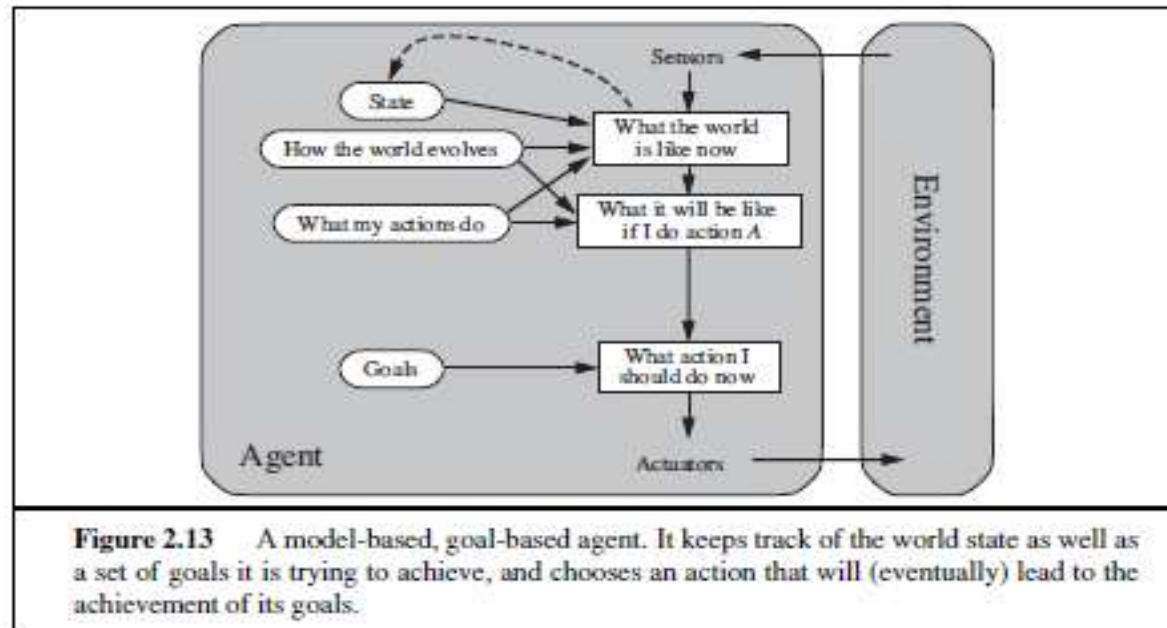
Figure 2.12 A model-based reflex agent. It keeps track of the current state of the world, using an internal model. It then chooses an action in the same way as the reflex agent.

Goal-based agent

- Uses information about the goal
- Search and planning are subfields of AI used to find action sequences that achieve the agent's goal
- This is more flexible than simple reflex agent

Ex: Google Maps Navigation, Provide many routes and chooses the best, re-route during traffic.

Goal-based agent (contd.)



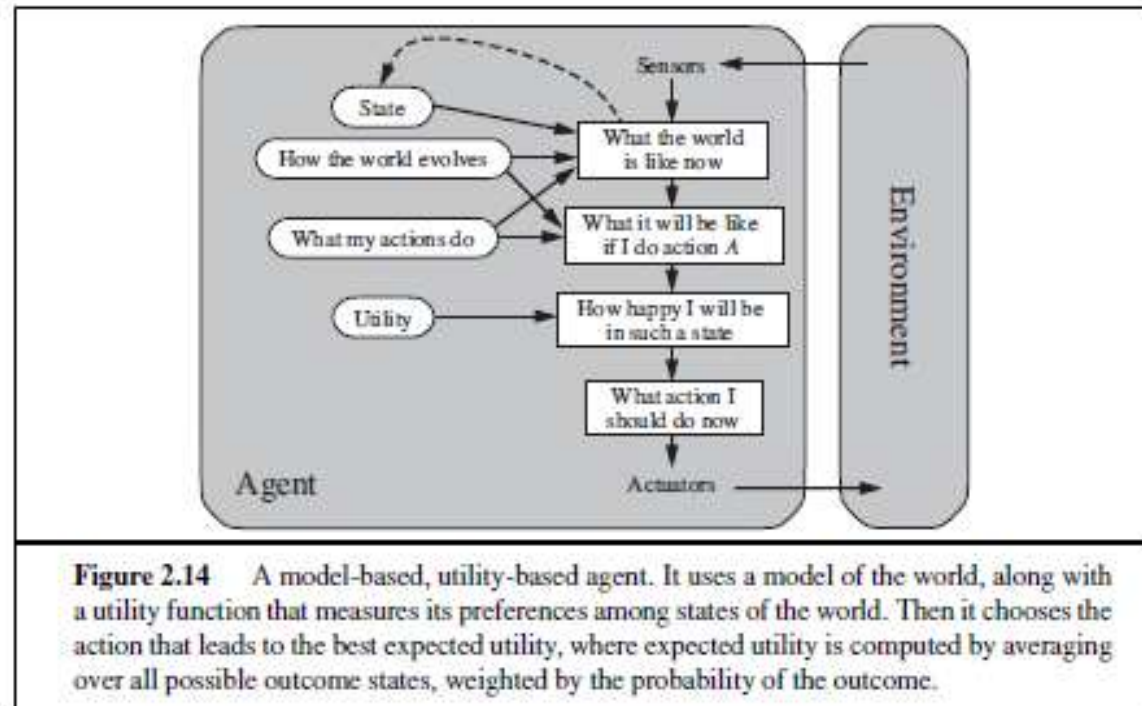
Utility-based agents

- Goals alone are not enough for high-quality behaviour
- E.g. many actions will get a person to the destination but some are quicker, safer, more reliable or cheaper than others
- A performance measure can be used depending on how happy it makes the agent called utility measure
- A rational utility-based agent chooses the action that maximises the agent's utility of the action outcome
- A utility-based agent has to model and keep track of its environment, which are tasks which involve perception, representation, reasoning and learning

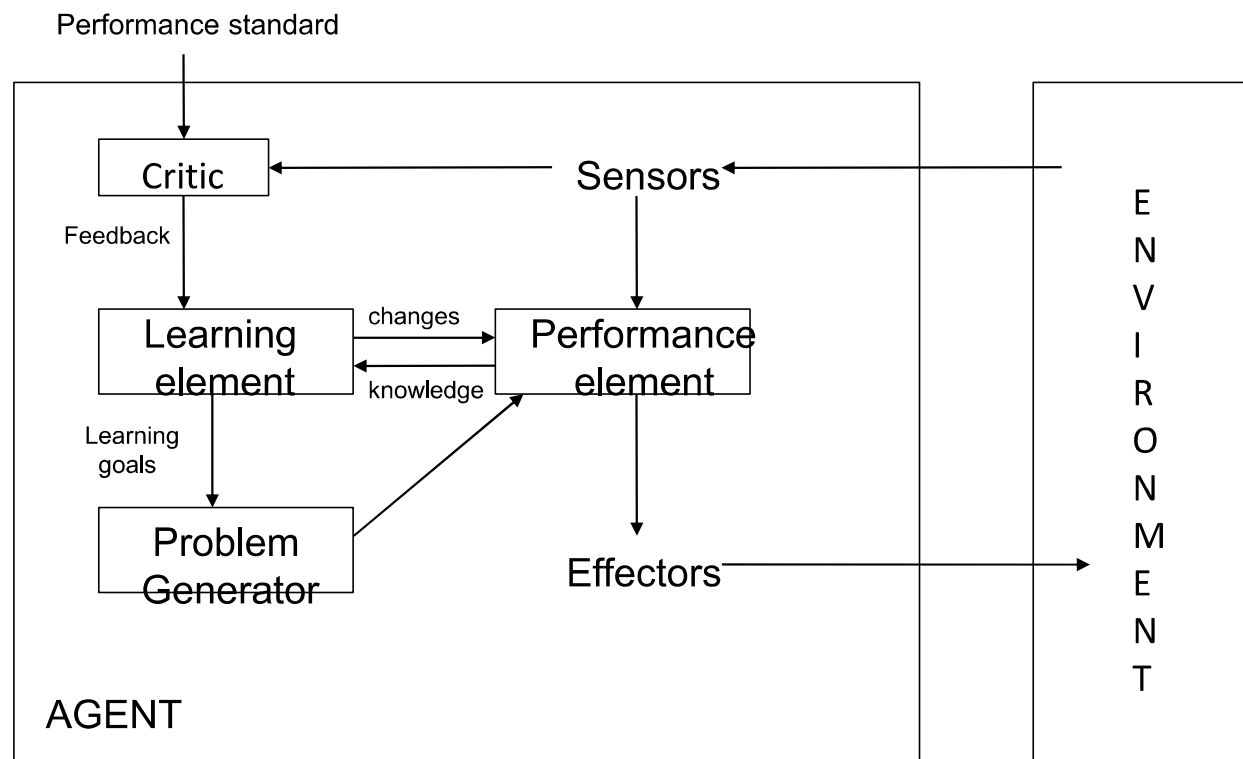
Ex: Self Driving Cars chooses routes with highest utility like fuel, traffic safety.

Food Delivery App utilities - Fast delivery, Low fuel cost, Happy customer, Traffic conditions

Utility-based agents (contd.)



Model of a learning agent



Task carried out by different components of learning agent

- ***Learning element*** : Makes improvements ; learning mechanism can be used to improve every part of the agent
- ***Performance element*** : Selects external actions
- ***Critic*** : Tells learning agent how well the agent is doing using a performance standard
- ***Problem generator*** : Suggests actions which lead to new and informative experiences